Web Technologies — Unit III & Unit IV Comprehensive Study Notes

> ***Subject Type:*** *Programming*
> ***Units Covered:*** *Unit III (PHP) & Unit IV (PHP & MySQL)*
> ***Exam Ready:*** *Yes*

Detailed Syllabus

UNIT-III (15 Marks)

PHP: Introduction to Server-Side Development with PHP, what is Server-Side Development, A Web Server's Responsibilities, Quick Tour of PHP, Introduction and basic syntax of PHP, decision and looping with examples, PHP and HTML, Arrays, Functions, Browser control and detection, string, Form processing, Files, Advance Features: Cookies and Sessions

---

UNIT-IV (15 Marks)

PHP and MySQL: Basic commands with PHP examples, Connection to the server, creating a database, selecting a database, listing database, listing table names, creating a table, inserting data, altering tables, queries, deleting the database, deleting data, and tables, PHP my admin and database bugs

State Management: Managing State, The Problem of State in Web Applications, Passing Information via Query Strings, Passing Information via the URL Path, Cookies, Serialization, Session State

---

Table of Contents

1. Unit III: PHP Server-Side Development
2. Unit IV: PHP and MySQL

Unit III: PHP Server-Side Development

1. Introduction to PHP

1.1 Definition and Overview

What is PHP?

PHP (PHP: Hypertext Preprocessor) is a widely-used, open-source server-side scripting language designed specifically for web development. Originally created by Rasmus Lerdorf in 1994, PHP has evolved into one of the most popular server-side technologies powering millions of websites worldwide.

Why PHP is Called "Hypertext Preprocessor":

The name "PHP" is a recursive acronym - PHP Hypertext Preprocessor. This means PHP processes (preprocesses) hypertext, typically HTML, on the server before sending it to the client.

How PHP Works:

1. User requests a PHP page in browser
2. Server receives the request
3. PHP engine interprets the PHP code
4. PHP executes server-side logic (database queries, calculations, etc.)
5. Server generates HTML response
6. HTML is sent to the client's browser
7. Client only sees the resulting HTML (not PHP code)

1.2 Why Server-Side Development?

Understanding Server-Side vs Client-Side:

Client-Side (Browser-based):

- HTML, CSS, JavaScript run in user's browser
- Code is visible to the client (security risk)
- Cannot directly access databases or file systems
- Examples: JavaScript, HTML, CSS

Server-Side:

- Code executes on the web server
- Code is hidden from clients (secure)
- Can access databases, files, other servers
- Examples: PHP, Python, Ruby, Java, Node.js

Benefits of Server-Side Development:

1. **Security:**
    - Sensitive code stays on server
    - Database credentials protected
    - Business logic hidden from users
    - Prevents tampering with functionality
2. **Database Access:**
    - Direct connection to databases
    - CRUD operations (Create, Read, Update, Delete)
    - Data validation and integrity
3. **Dynamic Content:**
    - Content changes based on user input
    - Personalized experiences
    - Real-time data display
4. **Session Management:**
    - Track user login state
    - Shopping carts
    - User preferences
5. **Performance:**
    - Heavy processing done on server
    - Client only receives rendered output
    - Reduces client device requirements

1.3 A Web Server's Responsibilities

A web server is the foundation of web hosting. Understanding its responsibilities helps in effective PHP development.

Core Responsibilities:

1. **HTTP Request Handling:**
    - Listen for incoming requests
    - Parse request headers and body
    - Route requests to appropriate handlers
    - Return appropriate responses

2. **Response Generation:**
    - Process server-side code (PHP, Python, etc.)
    - Generate dynamic HTML
    - Set appropriate headers
    - Compress responses for efficiency

3. **Static Content Delivery:**
    - Serve HTML, CSS, JavaScript files
    - Serve images, videos, documents
    - Cache static content for speed

4. **Security:**
    - SSL/TLS encryption
    - Authentication mechanisms
    - Firewall protection
    - DDoS mitigation

5. **Logging and Monitoring:**
    - Track visitor statistics
    - Error logging
    - Performance monitoring
    - Security auditing

6. **Resource Management:**
    - Memory management

- Concurrent connection handling
- Load balancing
- Session management

Popular Web Servers:

- Apache HTTP Server
- Nginx
- Microsoft IIS
- LiteSpeed

1.4 Basic Syntax

PHP code is embedded within HTML and executed on the server. Here's how it works:

```
<?php
// PHP code goes here
echo "Hello, World!";
?>

<!DOCTYPE html>
<html>
<body>
    <h1><?php echo "Welcome"; ?></h1>
    <?php $name = "John"; ?>
    <p>Hello, <?php echo $name; ?></p>
</body>
</html>
```

---

2. PHP Data Types and Variables

```php
<?php
// Variables
$name = "John";       // String
$age = 25;            // Integer
$price = 19.99;      // Float
$isActive = true;     // Boolean
$items = array("a", "b", "c");  // Array

// Object
class Person {
    public $name;
    public function greet() {
        return "Hello";
    }
}
$person = new Person();

// Null
$nullVar = null;

// Type casting
$num = (int)"42";     // Convert string to int
$double = (double)$num;
$bool = (bool)1;

// Type checking
gettype($name);       // "string"
is_array($items);    // true
is_int($age);        // true
?>
```

3. Operators in PHP

```php
<?php
// Arithmetic
$a + $b; $a - $b; $a * $b; $a / $b; $a % $b;

// Assignment
$a = 5; $a += 3; $a -= 2; $a *= 2; $a /= 2;

// Comparison
$a == $b;    // Equal
$a === $b;   // Identical (same type)
$a != $b;    // Not equal
$a <> $b;     // Not equal
$a !== $b;   // Not identical
$a > $b; $a < $b; $a >= $b; $a <= $b;

// Increment/Decrement
++$a; $a++; --$a; $a--;

// Logical
$a && $b; $a || $b; !$a;
$a and $b; $a or $b; $a xor $b;  // Different precedence

// Ternary
$status = ($age >= 18) ? "Adult" : "Minor";

// Null coalescing (PHP 7+)
$name = $_POST["name"] ?? "Guest";

// Spaceship (PHP 7+)
$a <=> $b;  // Returns -1, 0, or 1
?>
```

4. Control Structures in PHP

4.1 Decision Making

```php
<?php
// If-Else
if ($age >= 18) {
    echo "Adult";
} elseif ($age >= 13) {
    echo "Teenager";
} else {
    echo "Child";
}

// Ternary
$status = ($age >= 18) ? "Adult" : "Minor";

// Switch
switch ($day) {
    case "Monday":
        echo "Day 1";
        break;
    case "Tuesday":
        echo "Day 2";
        break;
    default:
        echo "Other day";
}

// Match (PHP 8+)
$result = match($day) {
    "Monday" => "Day 1",
    "Tuesday" => "Day 2",
    default => "Other day",
};
?>
```

4.2 Loops in PHP

```php
<?php
// For loop
for ($i = 0; $i < 5; $i++) {
    echo $i;
}

// While loop
while ($i < 5) {
    echo $i;
    $i++;
}

// Do-While
do {
    echo $i;
    $i++;
} while ($i < 5);

// Foreach (arrays)
$colors = ["red", "green", "blue"];
foreach ($colors as $color) {
    echo $color;
}

// Foreach with key
$data = ["name" => "John", "age" => 25];
foreach ($data as $key => $value) {
    echo "$key: $value";
}
?>
```

---

5. Arrays in PHP

```php
<?php
// Indexed arrays
$fruits = array("Apple", "Banana", "Orange");
// or
$fruits = ["Apple", "Banana", "Orange"];

echo $fruits[0];  // Apple
count($fruits);   // 3

// Associative arrays
$person = array(
    "name" => "John",
    "age" => 25,
    "city" => "New York"
);

echo $person["name"];  // John

// Multidimensional arrays
$students = array(
    array("John", 25),
    array("Jane", 22)
);
echo $students[0][0];  // John
?>
```

Array Functions:

```php
<?php
// Manipulation
array_push($arr, "new");      // Add element
array_pop($arr);             // Remove last
array_shift($arr);           // Remove first
array_unshift($arr, "new");  // Add to start

// Search
in_array("value", $arr);     // true/false
array_search("value", $arr); // index or false
array_key_exists("key", $arr);

// Sort
sort($arr);                  // Sort ascending
rsort($arr);                 // Sort descending
asort($arr);                 // Sort by value, keep keys
ksort($arr);                 // Sort by key

// Merge
array_merge($arr1, $arr2);   // Merge arrays

// Filter/Map
array_filter($arr, function($v) { return $v > 0; });
array_map(function($v) { return $v * 2; }, $arr);

// Other
count($arr);                 // Count elements
array_unique($arr);          // Remove duplicates
array_reverse($arr);         // Reverse array
?>
```

6. Functions in PHP

```php
<?php
// User-defined function
function greet($name) {
    return "Hello, $name";
}

echo greet("John");  // Hello, John

// Default parameters
function welcome($name = "Guest") {
    return "Welcome, $name";
}

// Passing by reference
function addFive(&$num) {
    $num += 5;
}
$value = 10;
addFive($value);
echo $value;  // 15

// Variable functions
$func = "greet";
echo $func("John");  // Calls greet()

// Anonymous function (closure)
$greet = function($name) {
    return "Hello, $name";
};
echo $greet("John");

// Arrow function (PHP 7.4+)
$double = fn($x) => $x * 2;
?>
```

String Functions:

```php
<?php
$str = "Hello World";
strlen($str);           // 11
strtoupper($str);      // HELLO WORLD
strtolower($str);      // hello world
substr($str, 0, 5);    // Hello
strpos($str, "World"); // 6
str_replace("World", "PHP", $str);  // Hello PHP
explode(" ", $str);    // ["Hello", "World"]
implode(" ", $arr);    // "Hello World"
trim($str);            // Remove whitespace
str_repeat("*", 5);   // "*****"
strrev($str);          // "dlroW olleH"
?>
```

7. Form Processing in PHP

7.1 HTML Form

```
<form action="process.php" method="post">
    Name: <input type="text" name="name"><br>
    Email: <input type="email" name="email"><br>
    Password: <input type="password" name="password"><br>
    <input type="submit" value="Submit">
</form>
```

7.2 PHP Form Handling

```
<?php
// Check if form submitted
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    // Get form values
    $name = $_POST["name"] ?? "";
    $email = $_POST["email"] ?? "";
    $password = $_POST["password"] ?? "";

    // Sanitize input
    $name = htmlspecialchars(trim($name));
    $email = filter_var($email, FILTER_SANITIZE_EMAIL);

    // Validate
    $errors = [];

    if (empty($name)) {
        $errors[] = "Name is required";
    }

    if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
        $errors[] = "Invalid email";
    }

    if (strlen($password) < 8) {
        $errors[] = "Password must be at least 8 characters";
    }

    if (empty($errors)) {
        echo "Form submitted successfully!";
    } else {
        foreach ($errors as $error) {
            echo "<p style='color:red'>$error</p>";
        }
    }
}
?>
```

---

8. File Handling in PHP

```php
<?php
// Read file
$content = file_get_contents("data.txt");
$lines = file("data.txt");
$file = fopen("data.txt", "r");
while (!feof($file)) {
    echo fgets($file);
}
fclose($file);

// Write file
file_put_contents("data.txt", "New content");
// or
$file = fopen("data.txt", "w");
fwrite($file, "Content");
fclose($file);

// Append
file_put_contents("data.txt", "More content", FILE_APPEND);

// Check if file exists
if (file_exists("data.txt")) {
    echo "File exists";
}

// Get file info
filesize("data.txt");
filemtime("data.txt");
is_readable("data.txt");
is_writable("data.txt");

// Delete file
unlink("data.txt");
?>
```

---

9. Cookies and Sessions in PHP

9.1 Cookies

```php
<?php
// Set cookie
setcookie("user", "John", time() + 3600, "/");
// or with options
setcookie("user", "John", [
    'expires' => time() + 3600,
    'path' => '/',
    'secure' => true,
    'httponly' => true,
    'samesite' => 'Strict'
]);

// Read cookie
if (isset($_COOKIE["user"])) {
    echo $_COOKIE["user"];
}

// Delete cookie
setcookie("user", "", time() - 3600);
?>
```

9.2 Sessions

```php
<?php
// Start session (must be at the top)
session_start();

// Set session variable
$_SESSION["username"] = "John";
$_SESSION["userid"] = 1;
$_SESSION["role"] = "admin";

// Check if logged in
if (isset($_SESSION["username"])) {
    echo "Welcome, " . $_SESSION["username"];
}

// Get all session data
print_r($_SESSION);

// Destroy specific session variable
unset($_SESSION["username"]);

// Destroy entire session
session_destroy();

// Regenerate session ID (security)
session_regenerate_id(true);
?>
```

---

10. Browser Control and Detection in PHP

```php
<?php
// Get user agent
$browser = $_SERVER["HTTP_USER_AGENT"];

// Get IP address
$ip = $_SERVER["REMOTE_ADDR"];

// Get current page
$page = $_SERVER["PHP_SELF"];

// Get referrer
$ref = $_SERVER["HTTP_REFERER"];

// Get request method
$method = $_SERVER["REQUEST_METHOD"];

// Browser detection
if (strpos($browser, "Firefox") !== false) {
    echo "Using Firefox";
} elseif (strpos($browser, "Chrome") !== false) {
    echo "Using Chrome";
} elseif (strpos($browser, "Safari") !== false) {
    echo "Using Safari";
}

// Get server info
echo $_SERVER["SERVER_NAME"];
echo $_SERVER["SERVER_PORT"];
echo $_SERVER["DOCUMENT_ROOT"];
?>
```

Unit III: Practice Questions

1. **[Easy]** Explain the difference between `echo` and `print` in PHP with examples.
2. **[Medium]** Create a PHP form that accepts user registration data (name, email, password, confirm password) with server-side validation and displays appropriate error messages.
3. **[Hard]** Write a PHP script that handles file upload with size (max 2MB) and type (images only: jpg, png, gif) validation, then saves the file to an uploads directory.

Unit III: Quick Reference Card

Concept	Syntax/Formula	Example
Variable	`$name`	`$name = "John"`
Array	`array()`	`["a", "b"]`
Function	`function name() {}`	Custom functions
Form GET	`$_GET["field"]`	URL parameters
Form POST	`$_POST["field"]`	Form data
Session	`$_SESSION["key"]`	Store user data
Cookie	`setcookie()`	Client storage
File	`file_get_contents()`	Read file
Superglobal	`$_SERVER`	Server info

Unit IV: PHP and MySQL

1. MySQL Basics

1.1 Introduction to MySQL

What is MySQL?

MySQL is the world's most popular open-source relational database management system (RDBMS). It uses SQL (Structured Query Language) for managing data and is a core component of the LAMP (Linux, Apache, MySQL, PHP/Python/Perl) stack.

Why MySQL is Important:

1. **Open Source:** Free to use, with large community support
2. **Performance:** Fast and efficient for most applications
3. **Scalability:** Handles small to enterprise-level databases

4. **Compatibility:** Works with PHP, Python, Java, and more
5. **Reliability:** Proven stability in production environments

MySQL Architecture:

1. **Client Layer:** Handles connection management, authentication
2. **Server Layer:** Query parsing, optimization, caching
3. **Storage Engine Layer:** Data storage and retrieval (InnoDB, MyISAM)

1.2 Connecting to MySQL

Connection Methods in PHP:

Method 1: MySQLi Procedural:

```php
<?php
$host = "localhost";
$user = "root";
$pass = "";
$dbname = "testdb";

// Establish connection
$conn = mysqli_connect($host, $user, $pass, $dbname);

// Check connection
if (!$conn) {
    die("Connection failed: " . mysqli_connect_error());
}
echo "Connected successfully";

// Close connection
mysqli_close($conn);
?>
```

Method 2: MySQLi Object-Oriented:

```php
<?php
$conn = new mysqli("localhost", "root", "", "testdb");

if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}
echo "Connected successfully";

$conn->close();
?>
```

Method 3: PDO (PHP Data Objects):

```php
<?php
try {
    $conn = new PDO("mysql:host=localhost;dbname=testdb", "root", "");
    $conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
    echo "Connected successfully";
} catch(PDOException $e) {
    echo "Connection failed: " . $e->getMessage();
}
?>
```

Which Method to Use?

Method	Pros	Cons
MySQLi Procedural	Simple, familiar	Less OOP
MySQLi OOP	Full OOP support	MySQL only
PDO	Database agnostic	Slightly complex

1.3 Database Operations

Creating a Database:

```php
<?php
$conn = new mysqli("localhost", "root", "");

// Create database
$sql = "CREATE DATABASE mywebapp";
if ($conn->query($sql) === TRUE) {
    echo "Database created successfully";
} else {
    echo "Error: " . $conn->error;
}

// Select database
$conn->select_db("mywebapp");

$conn->close();
?>
```

Creating Tables:

```php
<?php
$conn = new mysqli("localhost", "root", "", "mywebapp");

// Create users table
$sql = "CREATE TABLE users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    email VARCHAR(100) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,
    full_name VARCHAR(100),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    status ENUM('active', 'inactive') DEFAULT 'active'
) ENGINE=InnoDB";

if ($conn->query($sql) === TRUE) {
    echo "Table created successfully";
} else {
    echo "Error: " . $conn->error;
}
?>
```

Table Data Types:

Category	Types	Description
Numeric	INT, FLOAT, DOUBLE, DECIMAL	Whole and decimal numbers
String	CHAR, VARCHAR, TEXT	Character data
Date/Time	DATE, TIME, DATETIME, TIMESTAMP	Temporal data
Enum	ENUM	Predefined set of values
BLOB	BLOB, LONGBLOB	Binary data

2. CRUD Operations - Detailed

Create (INSERT):

```php
<?php
$conn = new mysqli("localhost", "root", "", "mywebapp");

// Method 1: Simple INSERT
$sql = "INSERT INTO users (username, email, password)
        VALUES ('john', 'john@example.com', 'hashed_pwd')";
$conn->query($sql);

// Method 2: Prepared Statement (RECOMMENDED)
$stmt = $conn->prepare("INSERT INTO users (username, email, password) VALUES (?, ?, ?)");
$stmt->bind_param("sss", $username, $email, $password);

$username = "jane";
$email = "jane@example.com";
$password = password_hash("password123", PASSWORD_DEFAULT);

$stmt->execute();
echo "Inserted ID: " . $stmt->insert_id;
$stmt->close();

// Method 3: Insert multiple rows
$users = [
    ['name1', 'email1@test.com', 'pass1'],
    ['name2', 'email2@test.com', 'pass2'],
    ['name3', 'email3@test.com', 'pass3']
];

$stmt = $conn->prepare("INSERT INTO users (username, email, password) VALUES (?, ?, ?)");
foreach ($users as $user) {
    $stmt->bind_param("sss", $user[0], $user[1], $user[2]);
    $stmt->execute();
}
?>
```

Read (SELECT):

```php
<?php
$conn = new mysqli("localhost", "root", "", "mywebapp");

// Method 1: Simple query
$result = $conn->query("SELECT * FROM users");

if ($result->num_rows > 0) {
    while ($row = $result->fetch_assoc()) {
        echo "ID: " . $row["id"] . " - Name: " . $row["username"] . "<br>";
    }
}

// Method 2: Prepared Statement
$stmt = $conn->prepare("SELECT * FROM users WHERE id = ?");
$stmt->bind_param("i", $userId);
$userId = 1;
$stmt->execute();
$result = $stmt->get_result();

while ($row = $result->fetch_assoc()) {
    echo $row["username"] . "<br>";
}

// Method 3: Query with conditions
$sql = "SELECT * FROM users WHERE status = 'active' ORDER BY created_at DESC LIMIT 10";
$result = $conn->query($sql);

// Method 4: COUNT, SUM, AVG
$sql = "SELECT COUNT(*) as total, AVG(price) as average FROM products";
$result = $conn->query($sql);
$row = $result->fetch_assoc();
echo "Total: " . $row["total"];

$stmt->close();
?>
```

Update:

```php
<?php
$conn = new mysqli("localhost", "root", "", "mywebapp");

// Method 1: Simple update
$sql = "UPDATE users SET email = 'newemail@test.com' WHERE id = 1";
$conn->query($sql);

// Method 2: Prepared Statement
$stmt = $conn->prepare("UPDATE users SET username = ?, email = ? WHERE id = ?");
$stmt->bind_param("ssi", $username, $email, $id);

$username = "john_updated";
$email = "john.new@example.com";
$id = 1;

$stmt->execute();
echo "Rows affected: " . $stmt->affected_rows;

// Method 3: Increment values
$sql = "UPDATE products SET views = views + 1 WHERE id = 5";
$conn->query($sql);

$stmt->close();
?>
```

Delete:

```php
<?php
$conn = new mysqli("localhost", "root", "", "mywebapp");

// Method 1: Simple delete
$sql = "DELETE FROM users WHERE id = 5";
$conn->query($sql);

// Method 2: Prepared Statement
$stmt = $conn->prepare("DELETE FROM users WHERE status = ? AND created_at < ?");
$stmt->bind_param("ss", $status, $date);

$status = "inactive";
$date = "2023-01-01";
$stmt->execute();

// Method 3: Delete all
$conn->query("DELETE FROM session_tokens");

$stmt->close();
?>
```

3. PHPMyAdmin

What is PHPMyAdmin?

phpMyAdmin is a free, open-source web-based tool for managing MySQL databases. It provides a graphical interface for database operations.

Key Features:

1. **Database Management:**
   - Create/delete databases
   - Manage users and privileges
   - Import/export databases
2. **Table Operations:**
   - Create/modify/delete tables
   - Add/modify/delete columns
   - Create indexes
3. **Data Operations:**
   - Browse data with filters
   - Add/edit/delete records
   - Search functionality
4. **SQL Operations:**
   - Execute SQL queries
   - Import SQL files
   - Export data

Common Database Bugs and Solutions:

Bug	Cause	Solution
"Table doesn't exist"	Wrong database selected	Use `USE database_name`
"Duplicate entry"	Unique constraint violation	Check unique fields
"Access denied"	Wrong credentials	Verify username/password
"Syntax error"	Invalid SQL	Check query syntax
"Lost connection"	Server timeout	Increase timeout
"Disk full"	No space	Free up disk space

Best Practices for Database Development:

1. **Use Prepared Statements** - Prevents SQL injection
2. **Validate Input** - Check data before queries
3. **Close Connections** - Always close database connections
4. **Use Transactions** - For multiple related operations
5. **Backup Regularly** - Prevent data loss
6. **Optimize Queries** - Use EXPLAIN for analysis
7. **Index Wisely** - Improve query performance

```php
<?php
$servername = "localhost";
$username = "root";
$password = "";
$database = "testdb";

// Method 1: MySQLi (procedural)
$conn = mysqli_connect($servername, $username, $password, $database);
if (!$conn) {
    die("Connection failed: " . mysqli_connect_error());
}
echo "Connected successfully";

// Method 2: MySQLi (object-oriented)
$conn = new mysqli($servername, $username, $password, $database);
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}

// Close connection
mysqli_close($conn);
// or
$conn->close();
?>
```

1.2 Creating Database and Table

```php
<?php
// Create database
$sql = "CREATE DATABASE mydb";
if ($conn->query($sql) === TRUE) {
    echo "Database created";
} else {
    echo "Error: " . $conn->error;
}

// Select database
$conn->select_db("mydb");

// Create table
$sql = "CREATE TABLE users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    password VARCHAR(255) NOT NULL,
    age INT DEFAULT 18,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
)";

if ($conn->query($sql) === TRUE) {
    echo "Table created";
}
?>
```

2. CRUD Operations

2.1 Insert Data

```php
<?php
// Prepared statement (recommended - prevents SQL injection)
$stmt = $conn->prepare("INSERT INTO users (name, email, password) VALUES (?, ?, ?)");
$stmt->bind_param("sss", $name, $email, $password);

$name = "John";
$email = "john@example.com";
$password = password_hash("password123", PASSWORD_DEFAULT);

$stmt->execute();
echo "Records inserted: " . $stmt->affected_rows;

// Direct insert (not recommended)
$sql = "INSERT INTO users (name, email) VALUES ('Jane', 'jane@example.com')";
$conn->query($sql);

// Insert multiple
$sql = "INSERT INTO users (name, email) VALUES (?, ?), (?, ?)";
$stmt = $conn->prepare($sql);
$stmt->bind_param("ssss", $name1, $email1, $name2, $email2);
$name1 = "John"; $email1 = "john@example.com";
$name2 = "Jane"; $email2 = "jane@example.com";
$stmt->execute();
?>
```

2.2 Select Data

```php
<?php
// Fetch all results
$sql = "SELECT * FROM users";
$result = $conn->query($sql);

if ($result->num_rows > 0) {
    // Method 1: Associative array
    while($row = $result->fetch_assoc()) {
        echo "ID: " . $row["id"] . " - Name: " . $row["name"];
    }

    // Method 2: All at once
    $rows = $result->fetch_all(MYSQLI_ASSOC);
    print_r($rows);
} else {
    echo "No results";
}

// Using WHERE
$stmt = $conn->prepare("SELECT * FROM users WHERE id = ?");
$stmt->bind_param("i", $id);
$id = 1;
$stmt->execute();
$result = $stmt->get_result();

// Select specific columns
$sql = "SELECT name, email FROM users";
?>
```

2.3 Update Data

```php
<?php
// Update record
$stmt = $conn->prepare("UPDATE users SET name = ?, email = ? WHERE id = ?");
$stmt->bind_param("ssi", $name, $email, $id);

$name = "John Updated";
$email = "john.new@example.com";
$id = 1;

$stmt->execute();
echo $stmt->affected_rows . " record(s) updated";

// Update with WHERE condition
$sql = "UPDATE users SET age = age + 1 WHERE status = 'active'";
$conn->query($sql);
?>
```

2.4 Delete Data

```php
<?php
// Delete record
$stmt = $conn->prepare("DELETE FROM users WHERE id = ?");
$stmt->bind_param("i", $id);
$id = 1;
$stmt->execute();

// Delete with condition
$sql = "DELETE FROM users WHERE status = 'inactive' AND created_at < '2023-01-01'";
$conn->query($sql);

// Delete all
$conn->query("DELETE FROM users");
?>
```

---

3. Managing State in Web Applications

3.1 The Problem of State

Understanding the Stateless Nature of HTTP:

HTTP (Hypertext Transfer Protocol) is fundamentally stateless. This means each request from a client to a server is independent - the server doesn't remember previous requests from the same client. This creates a challenge for web applications that need to maintain user sessions, track shopping carts, or remember user preferences.

Why State Management is Challenging:

1. **Every Request is Independent:**
   - Server doesn't know if two requests are from the same user
   - No built-in memory of previous interactions
   - Makes personalization difficult
2. **Authentication Problems:**
   - Users would need to login for every single action
   - No way to remember logged-in users
3. **Shopping Carts Impossible:**
   - E-commerce sites couldn't track items

    - No continuity between page visits

4. **User Experience Degradation:**
    - Forced to re-enter information
    - No personalized content

The Solution:

Web developers use various techniques to maintain state:

- Sessions
- Cookies
- Database storage
- Hidden form fields
- URL parameters

3.2 Detailed State Management Methods

Method 1: Query Strings

Query strings append data to URLs in the form of key-value pairs after a question mark.

How It Works:

```
URL: example.com/product.php?id=123&category=electronics
```

Advantages:

- Simple to implement
- Works without cookies
- Easy to bookmark/share
- Server-side only (secure)

Disadvantages:

- Visible in URL (privacy concern)
- Limited length (~2000 characters)
- User can bookmark/copy URL (security risk)
- Not suitable for sensitive data

When to Use:

- Passing non-sensitive data between pages
- Search results
- Pagination
- Filter parameters

Example:

```
<!-- HTML Link -->
<a href="product.php?id=123">View Product</a>

<!-- PHP Handling -->
<?php
$productId = $_GET["id"];
$category = $_GET["category"];
?>
```

Method 2: URL Path Information

Modern RESTful URLs embed parameters directly in the path.

How It Works:

```
URL: example.com/users/123/profile
```

Advantages:

- Cleaner URLs (SEO-friendly)
- User-friendly and memorable
- Better for search engines

Disadvantages:

- Requires URL rewriting (Apache mod_rewrite)
- More complex implementation
- Still visible in URL

Example:

```php
<?php
// URL: example.com/user/123
$path = $_SERVER["REQUEST_URI"];
$parts = explode("/", $path);
// $parts[2] would be "123"
$userId = $parts[2];
?>
```

Method 3: Hidden Form Fields

Data is stored in hidden input fields and submitted with forms.

How It Works:

```html
<form action="process.php" method="post">
    <input type="hidden" name="user_id" value="123">
    <input type="hidden" name="cart_total" value="99.99">
    <button type="submit">Checkout</button>
</form>
```

Advantages:

- Hidden from URL
- Can pass larger amounts of data
- Works without cookies

Disadvantages:

- Only works with form submissions
- View Source reveals data
- Can't be bookmarked

When to Use:

- Multi-step forms
- Passing data between pages via forms

Method 4: Cookies

Cookies are small pieces of data stored on the client's browser by the server.

How Cookies Work:

1. Server sends Set-Cookie header in response
2. Browser stores the cookie
3. Browser sends cookie with every subsequent request
4. Server reads cookie to identify user

Types of Cookies:

1. **Session Cookies:**
   - Temporary, deleted when browser closes
   - No expiration date set
   - Used for basic session tracking
2. **Persistent Cookies:**
   - Remain until expiration date
   - Survives browser closure
   - Used for "Remember Me" features

Cookie Structure:

```
name=value; expires=date; path=/; domain=example.com; secure; HttpOnly
```

Cookie Security:

Pros:

- Maintain login state
- Remember preferences
- Track user behavior (analytics)

Cons:

- Can be viewed/modified by users
- Can be stolen (XSS attacks)
- Size limit (~4KB)
- Users can disable cookies
- Privacy concerns

Example:

```php
<?php
// Set cookie (expires in 1 hour)
setcookie("username", "john", time() + 3600, "/");

// Read cookie
if (isset($_COOKIE["username"])) {
    echo "Welcome, " . $_COOKIE["username"];
}

// Delete cookie
setcookie("username", "", time() - 3600);
?>
```

Method 5: PHP Sessions

Sessions provide a more secure way to store user data on the server.

How Sessions Work:

1. User visits site → Server creates unique session ID
2. Session ID stored in cookie (or URL)
3. Server stores session data in files/memory/database
4. Subsequent requests include session ID
5. Server retrieves associated data

Why Sessions are Better than Cookies:

Aspect	Cookies	Sessions
Storage	Client (browser)	Server
Capacity	~4KB	Unlimited
Security	Less secure	More secure
Data Type	Strings only	Any data type
Expiration	Manual control	Automatic

Session Implementation:

```php
<?php
// Start session (must be first output)
session_start();

// Set session data
$_SESSION["user_id"] = 123;
$_SESSION["username"] = "john";
$_SESSION["is_logged_in"] = true;

// Access session data
echo "Welcome, " . $_SESSION["username"];

// Destroy session
session_destroy();
?>
```

Session Security Best Practices:

1. Regenerate session ID after login:

```php
session_regenerate_id(true);
```

2. Store sensitive data in session, not cookies
3. Set session cookie as HttpOnly and Secure:

```php
ini_set("session.cookie_httponly", 1);
ini_set("session.cookie_secure", 1);
```

4. Set session timeout:

```php
ini_set("session.gc_maxlifetime", 1800); // 30 minutes
```

Method 6: Serialization

Serialization converts complex data structures (arrays, objects) into a storable format.

PHP Serialization:

```php
<?php
// Convert array to string
$data = ["name" => "John", "age" => 25, "city" => "NYC"];
$serialized = serialize($data);
// $serialized = 'a:3:{s:4:"name";s:4:"John";s:3:"age";i:25;s:4:"city";s:3:"NYC";}'

// Convert back to array
$original = unserialize($serialized);

// JSON (preferred for web)
$json = json_encode($data);
$original = json_decode($json, true);
?>
```

When to Use Serialization:

- Store complex data in single cookie
- Save session data to database
- Cache data
- API data transmission

3.3 State Management Comparison

Method	Storage Location	Security	Capacity	Persistence
Query String	URL	Low	2KB	Until link shared
Hidden Fields	Request	Medium	Unlimited	One request
Cookies	Client Browser	Low	4KB	Manual expiry
Sessions	Server	High	Unlimited	Until expiry
Database	Server	High	Unlimited	Permanent

3.4 Choosing the Right Method

Use Query Strings When:

- Non-sensitive data
- Bookmarking needed
- Simple state transfer

Use Cookies When:

- Long-term preferences
- Analytics/tracking
- Simple client-side storage

Use Sessions When:

- User authentication
- Shopping carts
- Sensitive data
- Large amounts of data

Use Database When:

- Permanent storage needed
- Multiple servers (shared state)
- Complex queries on data

1. Query Strings:

```
<!-- HTML -->
<a href="page.php?id=123">Next Page</a>

<!-- PHP -->
$id = $_GET["id"];
```

2. URL Path:

```
// Rewrite URL example.com/user/123
$path = $_SERVER["REQUEST_URI"];
$parts = explode("/", $path);
$id = $parts[2];
```

3. Hidden Form Fields:

```
<input type="hidden" name="userid" value="123">
```

4. Cookies:

```php
<?php
setcookie("user", "123", time() + 3600);
$user = $_COOKIE["user"];
?>
```

5. Sessions:

```php
<?php
session_start();
$_SESSION["user"] = "John";
?>
```

6. PHP Serialization:

```php
<?php
$data = array("name" => "John", "age" => 25);
$serialized = serialize($data);
// Store in cookie/database
$unserialized = unserialize($serialized);

// JSON (preferred)
$json = json_encode($data);
$decoded = json_decode($json, true);
?>
```

---

4. PHPMyAdmin

- Web-based MySQL management tool
- Create/edit/delete databases, tables
- Execute SQL queries
- Manage users and permissions
- Import/export databases
- Database backup and restore
- Handle database bugs

Unit IV: Practice Questions

1. **[Easy]** Explain the difference between GET and POST methods in PHP. When should each be used?
2. **[Medium]** Create a PHP login system that validates user credentials against a MySQL database. Use prepared statements to prevent SQL injection.
3. **[Hard]** Design a complete CRUD application in PHP with proper session management, password hashing (bcrypt), and security measures against SQL injection, XSS, and CSRF attacks.

Unit IV: Quick Reference Card

Concept	Syntax/Formula	Example
Connect	`new mysqli()`	Server connection
Insert	`INSERT INTO`	Add records
Select	`SELECT * FROM`	Fetch records
Update	`UPDATE table SET`	Modify records
Delete	`DELETE FROM`	Remove records
Prepare	`->prepare()`	SQL security
Session	`$_SESSION`	State management
Cookie	`setcookie()`	Client storage
Hash	`password_hash()`	Password security
Query String	`$_GET["id"]`	URL parameters
Serialization	`serialize()`	Store complex data

Unit III & IV Summary

PHP vs MySQL Integration

Operation	MySQLi Procedural	MySQLi Object-Oriented
Connect	`mysqli_connect()`	`new mysqli()`
Query	`mysqli_query()`	`$conn->query()`
Insert	`mysqli_insert_id()`	`$conn->insert_id`
Close	`mysqli_close()`	`$conn->close()`

State Management Comparison

Method	Storage Location	Security	Capacity
Query String	URL	Low (visible)	Limited
Hidden Fields	Request	Medium	Limited
Cookies	Client Browser	Low	Small
Sessions	Server	High	Large
Database	Server	High	Unlimited

Security Best Practices

1. **SQL Injection:** Use prepared statements
2. **XSS:** Use `htmlspecialchars()` for output
3. **CSRF:** Use tokens for forms
4. **Passwords:** Use `password_hash()` and `password_verify()`

End of Document